

GeoCore

Complete Master Blueprint

From strategic vision to a scalable, locally relevant geospatial technology platform.

North Star:

Build reusable geospatial technology that can serve government, businesses, NGOs and individuals through a shared platform—without being dependent on a single proprietary GIS ecosystem.

Purpose of this document: This is the complete reference blueprint for the GeoCore journey. It combines the product vision, platform strategy, architecture, technology stack, database model, user structure, development roadmap and final product ecosystem into one document.

Master Reference Document — Version 1.0

Table of Contents

1	The Strategic Vision
2	The Problem We Are Solving
3	The Product Philosophy
4	The GeoCore Platform Concept
5	The Complete Platform Ecosystem
6	Core User Workflow
7	Multi-Tenant Architecture
8	System Architecture
9	Recommended Technology Stack
10	Core Database Architecture
11	Dynamic Geospatial Data Model
12	Forms and Field Data Collection
13	User Roles and Permissions
14	Core Platform Modules
15	API Architecture
16	Frontend Application Structure
17	Mobile and Field Collection Strategy
18	Reporting and Analytics
19	Security and Data Isolation
20	Development Roadmap
21	MVP Scope
22	What Not to Build Yet
23	Commercial and Scalability Strategy
24	Final Product Vision
25	The Actual Building Sequence

1. The Strategic Vision

The long-term goal is to move from delivering individual GIS projects to building reusable geospatial software products. Instead of building a completely different system for every organisation, the same core technology should be configured for different sectors and use cases.

Old model: Build a GIS solution for one client.

New model: Build a reusable geospatial platform that solves a recurring problem for many organisations.

The objective is not to recreate every feature of large enterprise GIS platforms. The objective is to build practical, locally relevant technology that solves real operational problems simply, securely and at scale.

2. The Problem We Are Solving

Many organisations need to collect, manage, visualise and report on location-based information, but they often face one or more of the following problems:

- Field data is collected using disconnected tools.
- Data is stored in spreadsheets, phones or separate systems.
- Organisations cannot easily visualise their information on maps.
- Reports are created manually and repeatedly.
- GIS software can be expensive or unnecessarily complex for a specific workflow.
- Different departments use incompatible systems.
- A solution built for one project cannot easily be reused for another.

GeoCore aims to connect the complete workflow: Define → Collect → Store → Map → Analyse → Report → Integrate.

3. The Product Philosophy

The core principle is simple:

Build the platform once. Configure it many times.

A drainage inventory, school inventory, road inventory, property inventory and borehole inventory may appear to be different projects. However, the underlying workflow is often similar.

Common Workflow	What the Platform Provides
Define the project	Project workspace and configuration
Define the data	Asset types and dynamic fields
Collect information	Forms, GPS, photos and field workflows
Store information	Spatial database and file storage
View information	Interactive maps and layers
Understand information	Charts, indicators and spatial analysis
Communicate results	Reports and exports

4. The GeoCore Platform Concept

GeoCore is the reusable core platform. Sector-specific products should be built on top of it rather than becoming entirely separate applications.



5. The Complete Platform Ecosystem

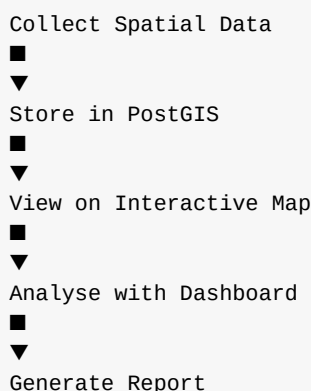
The long-term product family can be organised as a shared platform with specialised modules.

Product / Module	Purpose
GeoCore	Shared platform foundation: users, organisations, projects, spatial data, APIs and security.
GeoSurvey	Configurable forms and field data collection.
GeoAsset	Infrastructure and physical asset management.
GeoEstate	Property, land and real estate workflows.
GeoWorks	Project monitoring and field operations.
GeoGov	Government inventories, monitoring and operational workflows.
GeoAI	AI-assisted classification, analysis and decision support.

6. Core User Workflow

The first complete workflow should be:

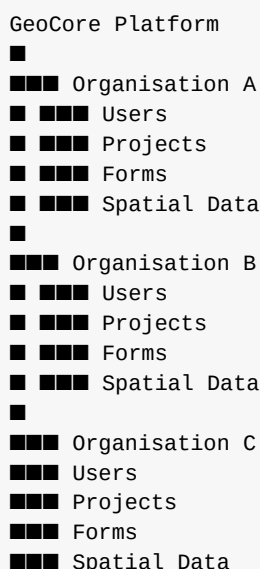




Example: an organisation creates a Drainage Inventory project, defines the drainage fields, collects GPS and photographs, views the drainage network on a map, analyses condition and generates a report.

7. Multi-Tenant Architecture

The platform should be designed from the beginning to support multiple organisations.



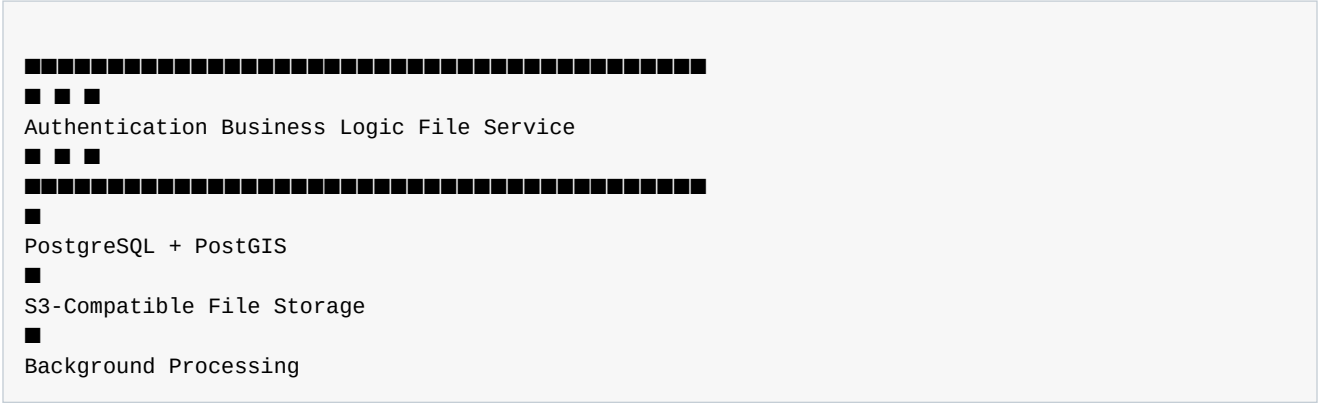
Critical security principle: Organisation A must never be able to access Organisation B's data unless an explicit, secure sharing mechanism is implemented.

Every organisation-scoped request must be checked against the authenticated user's membership and permissions.

8. System Architecture

The recommended high-level architecture is:





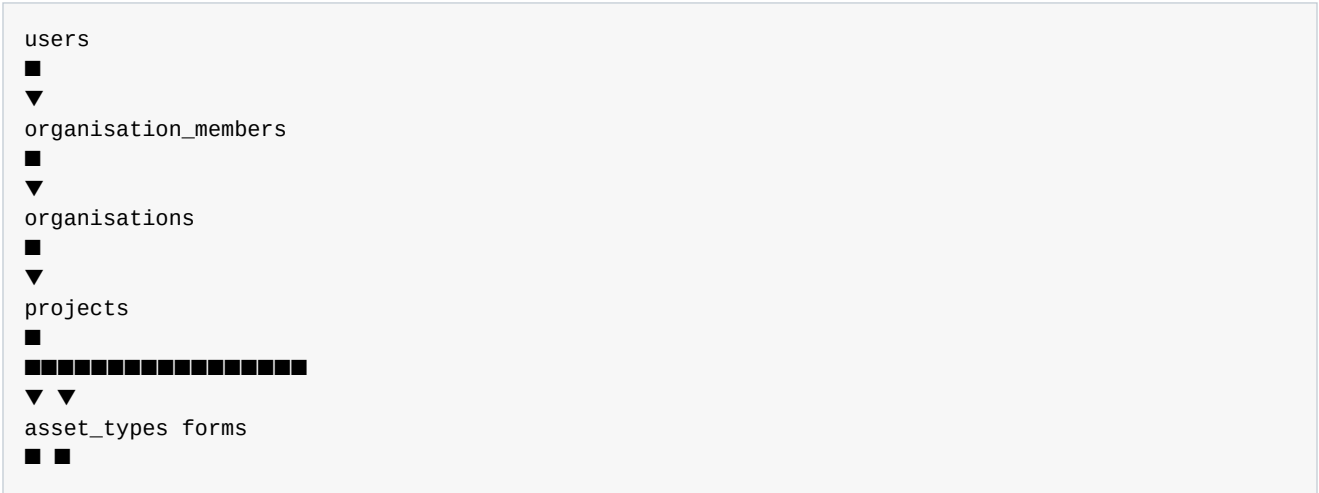
The frontend and mobile application should communicate with the same API. This ensures that the business rules, permissions and data validation remain centralised.

9. Recommended Technology Stack

Layer	Recommended Technology	Reason
Frontend	React + TypeScript	Reusable, scalable component architecture
Styling	Tailwind CSS	Fast, consistent UI development
Web Mapping	MapLibre GL JS or OpenLayers	Open mapping stack and strong geospatial capability
Backend	FastAPI + Python	Excellent API performance and geospatial ecosystem
Database	PostgreSQL + PostGIS	Strong relational and spatial database foundation
Validation	Pydantic	Structured API validation and schemas
ORM / DB Access	SQLAlchemy	Flexible database access and migrations
File Storage	S3-compatible storage	Scalable photos, documents and attachments
Background Jobs	Redis + task queue	Reports and long-running processing
Geospatial Processing	GDAL, GeoPandas, Rasterio	Open-source spatial processing
Containerisation	Docker	Consistent development and deployment
Version Control	Git + GitHub	Collaboration and source control

10. Core Database Architecture

The database should be designed around a small number of reusable entities rather than separate tables for every sector.



▼ ▼
 field_definitions form_fields
 ■
 ▼
 records
 ■
 ▼
 attachments

Entity	Purpose
users	User identity and authentication
organisations	Customer / tenant boundary
organisation_members	User membership and roles
projects	Operational workspaces
project_members	Project access control
asset_types	Reusable spatial object definitions
field_definitions	Configurable fields
forms	Data collection structures
form_fields	Fields included in a form
records	Actual spatial data
attachments	Photos and documents
reports	Generated outputs

11. Dynamic Geospatial Data Model

The system should not be hardcoded to only support roads, buildings or drainage. Users should be able to define their own asset types.

Asset Type	Possible Fields
Drainage	Type, condition, width, depth, material, photograph
School	Name, classrooms, condition, student population
Borehole	Depth, functionality, water quality, pump type
Road	Surface, condition, width, length, photograph
Property	Address, use, value, ownership status, photograph

The application remains the same. The configuration changes.

12. Forms and Field Data Collection

The form system is one of the most important parts of the platform. It should allow users to define how information is collected without requiring a developer to create a new form for every project.

Field Type	Example
Short Text	Asset name
Long Text	Description of damage
Number	Width, depth, population
Date	Inspection date
Date and Time	Incident timestamp
Single Select	Condition: Good / Fair / Poor
Multiple Select	Observed defects
Boolean	Is the asset functional?
GPS Location	Point coordinate
Photo	Evidence photograph
Video	Inspection video
File	Supporting document
Signature	Officer / respondent signature

Future capabilities can include conditional logic, repeat groups, calculations, validation rules, offline workflows and XLSForm-style import/export.

13. User Roles and Permissions

Role	Typical Permissions
Owner	Full organisation control and billing / account ownership
Administrator	Manage users, projects and organisation settings
Project Manager	Manage assigned projects and data workflows
Data Collector	Collect and update assigned field data
Analyst	View, filter, analyse and export data
Viewer	Read-only access

Permissions should be implemented at both organisation and project levels.

14. Core Platform Modules

Module	Initial Capability
Authentication	Register, login, password reset, tokens
Organisation Management	Create organisations, invite members, roles
Project Management	Create projects and manage project members
Asset Types	Define reusable categories of spatial data
Field Definitions	Define configurable fields and validation

Module	Initial Capability
Forms	Configure data collection workflows
Records	Create, update, query and delete spatial data
Map	Display points, lines, polygons and popups
Attachments	Store photos and documents
Dashboard	Indicators, charts and tables
Reports	Generate PDF and structured outputs

15. API Architecture

The API is the central contract between the frontend, mobile application and backend.

Method	Endpoint	Purpose
POST	/auth/register	Create user
POST	/auth/login	Authenticate user
GET	/organisations	List accessible organisations
POST	/organisations	Create organisation
POST	/organisations/{id}/members	Add / invite member
GET	/projects	List accessible projects
POST	/projects	Create project
GET	/projects/{id}	Get project details
POST	/asset-types	Create asset type
POST	/forms	Create form
POST	/records	Create spatial record
GET	/records	Query records
POST	/attachments	Upload attachment
POST	/reports	Generate report

16. Frontend Application Structure

```

apps/web
  ■■■ src/
  ■ ■■■ app/
  ■ ■■■ components/
  ■ ■■■ features/
  ■ ■ ■■■ auth/
  ■ ■ ■■■ organisations/
  ■ ■ ■■■ projects/
  ■ ■ ■■■ forms/
  ■ ■ ■■■ records/
  ■ ■ ■■■ maps/
  ■ ■ ■■■ reports/
  ■ ■■■ layouts/
  ■ ■■■ services/

```

- ■■■ hooks/
- ■■■ types/
- ■■■ routes/

Route	Purpose
/login	Authentication
/dashboard	Organisation overview
/projects	Project list
/projects/:id	Project overview
/projects/:id/forms	Form builder
/projects/:id/data	Record table
/projects/:id/map	Interactive map
/projects/:id/reports	Report management
/settings	Settings

17. Mobile and Field Collection Strategy

The field collection experience should eventually support mobile users, because many geospatial workflows occur outside the office.

The mobile strategy should evolve in stages:

Stage	Approach
MVP	Responsive web form usable on mobile devices
Next	Progressive Web App with caching and improved field experience
Later	Dedicated Android / cross-platform mobile application
Advanced	Offline-first synchronisation and background upload

Do not build a complex native mobile application before proving the core data workflow.

18. Reporting and Analytics

Reporting should be generated from the same data that powers the map and dashboard.

- Spatial Records
 -
 - Map
 -
 - Indicators
 -
 - Charts
 -
 - PDF Report

A report can contain project information, summary indicators, maps, charts, tables, photographs and narrative summaries.

The reporting engine should eventually become reusable across GeoSurvey, GeoAsset, GeoEstate and GeoGov.

19. Security and Data Isolation

- Use secure password hashing.
- Use token-based authentication with expiration and refresh strategy.
- Validate every organisation and project access request.
- Never rely solely on frontend permissions.
- Use database constraints to protect relationships.
- Store files securely and control access to attachments.
- Maintain audit information for important changes.
- Separate development, staging and production environments.
- Back up the database and define recovery procedures.

Security is not a feature to add at the end. It is part of the platform architecture.

20. Development Roadmap

Phase	Main Objective	Output
Phase 0	Planning	Architecture, schema and repository structure
Phase 1	Foundation	API, database, authentication
Phase 2	Tenancy	Organisations, members and roles
Phase 3	Projects	Project creation and access control
Phase 4	Dynamic Data	Asset types, fields and forms
Phase 5	Spatial Records	Create and manage points, lines and polygons
Phase 6	Mapping	Interactive map and filters
Phase 7	Attachments	Photos and supporting files
Phase 8	Dashboard	Indicators, charts and tables
Phase 9	Reports	PDF and structured reports
Phase 10	Pilot	Real-world use case and feedback

21. MVP Scope

The first MVP should be intentionally focused.

MVP Success Definition: A user can create an organisation, create a project, define a spatial asset, collect records and view those records on a map.

Must Have	Later
Authentication	Advanced SSO
Organisation management	Enterprise directory integrations
Projects	Complex portfolio management
Asset types	Advanced ontology systems

Must Have	Later
Dynamic fields	Every possible field type
Spatial records	Advanced 3D data
Basic map	Full enterprise cartography suite
Photo upload	Advanced media processing
Basic dashboard	Full drag-and-drop dashboard designer
Basic reports	Advanced automated intelligence reports

22. What Not to Build Yet

- A complete replacement for every enterprise GIS product.
- Advanced AI before the core data workflow is stable.
- Complex remote sensing processing in the first MVP.
- Dozens of integrations before the API is proven.
- A complex native mobile application before mobile workflows are validated.
- A massive dashboard builder before users need it.
- Every possible industry-specific module at once.

Start narrow. Build a strong foundation. Expand based on real users and real problems.

23. Commercial and Scalability Strategy

The platform can eventually support several commercial models.

Model	Description
SaaS Subscription	Organisations pay for hosted access to the platform.
Implementation	Custom configuration, setup and deployment services.
Enterprise Deployment	Private or dedicated deployments for larger organisations.
Sector Modules	Specialised products such as GeoEstate or GeoAsset.
Professional Services	Data collection, migration, training and analysis.
API / Integration	Connect GeoCore to other systems.

The strategic advantage is that services can generate revenue while the reusable platform gradually becomes the core product asset.

24. Final Product Vision



GeoCore



Maps Analytics Reports



Open Geospatial Foundation

The long-term transformation is from GIS service provider to geospatial product builder and platform owner.

The goal is not to build one application. The goal is to build a platform capable of producing many geospatial solutions.

25. The Actual Building Sequence

After this master blueprint, the planning phase is complete enough to begin implementation.

Order	Action
1	Create the GeoCore repository structure
2	Set up local development environment
3	Create PostgreSQL + PostGIS database
4	Create environment configuration
5	Create database migrations
6	Implement authentication
7	Implement organisations and memberships
8	Implement projects
9	Implement asset types and field definitions
10	Implement spatial records
11	Build first map interface
12	Add attachments
13	Add dashboard indicators
14	Add basic reporting
15	Pilot with a real geospatial use case

Next action: We now stop producing strategy documents and begin the actual building process. The first technical task should be creating the GeoCore project structure and development environment.

The first implementation should be treated as a real product foundation. Every feature should be evaluated against one question: **Can this be reused across multiple geospatial solutions?**

Final Reference: The GeoCore North Star

Build once. Configure many times.

Collect location-based data. Manage it securely. Visualise it clearly. Analyse it intelligently. Report it professionally. Reuse the platform across sectors.